

HỆ CƠ SỞ DỮ LIỆU Database Systems

Giảng viên: ThS. Đào Thị Ngọc Hân
Email: handtn@huce.edu.vn
SĐT: 0987-999-338

Bộ môn Công nghệ phần mềm – Phòng 408A1
Khoa Công nghệ thông tin
Đại học Xây dựng Hà Nội

1

CHƯƠNG 5

Giới thiệu Structured Query Language (SQL)

Slide có dùng các hình ảnh lấy từ Internet

2

Learning Objectives

- In this chapter, you will learn:
 - The basic **commands** and **functions** of SQL
 - How to use SQL for **data manipulation** (to *retrieve, modify, delete, and add* data)
 - How to use SQL for **data administration** (to create tables and indexes)
 - How to use SQL to **query** a database for useful information

3

3

SQL functions

- Data Manipulation Language (DML):
 - insert, update, delete, and retrieve data
- Data Definition Language (DDL)
 - create database objects such as tables, indexes, and views...
- Transaction Control Language (TCL)
- Data Control Language (DCL)

4

1. Structured Query Language (SQL)

- Nonprocedural language with basic command vocabulary set of **less than 100 words**
- Differences in SQL dialects are **minor**

5

5

SQL data manipulation commands

SQL DATA MANIPULATION COMMANDS	
COMMAND, OPTION, OR OPERATOR	DESCRIPTION
SELECT	Selects attributes from rows in one or more tables or views
FROM	Specifies the tables from which data should be retrieved
WHERE	Restricts the selection of rows based on a conditional expression
GROUP BY	Groups the selected rows based on one or more attributes
HAVING	Restricts the selection of grouped rows based on a condition
ORDER BY	Orders the selected rows based on one or more attributes
INSERT	Inserts row(s) into a table
UPDATE	Modifies an attribute's values in one or more table's rows
DELETE	Deletes one or more rows from a table
Comparison operators	
=, <, >, <=, >=, <>, !=	Used in conditional expressions
Logical operators	
AND/OR/NOT	Used in conditional expressions

6

SQL data manipulation commands

Special operators	Used in conditional expressions
BETWEEN	Checks whether an attribute value is within a range
IN	Checks whether an attribute value matches any value within a value list
LIKE	Checks whether an attribute value matches a given string pattern
IS NULL	Checks whether an attribute value is null
EXISTS	Checks whether a subquery returns any rows
DISTINCT	Limits values to unique values
Aggregate functions	Used with SELECT to return mathematical summaries on columns
COUNT	Returns the number of rows with non-null values for a given column
MIN	Returns the minimum attribute value found in a given column
MAX	Returns the maximum attribute value found in a given column
SUM	Returns the sum of all values for a given column
AVG	Returns the average of all values for a given column

7

SQL data definition commands

SQL DATA DEFINITION COMMANDS	
COMMAND OR OPTION	DESCRIPTION
CREATE SCHEMA AUTHORIZATION	Creates a database schema
CREATE TABLE	Creates a new table in the user's database schema
NOT NULL	Ensures that a column will not have null values
UNIQUE	Ensures that a column will not have duplicate values
PRIMARY KEY	Defines a primary key for a table
FOREIGN KEY	Defines a foreign key for a table
DEFAULT	Defines a default value for a column (when no value is given)
CHECK	Validates data in an attribute
CREATE INDEX	Creates an index for a table
CREATE VIEW	Creates a dynamic subset of rows and columns from one or more tables
ALTER TABLE	Modifies a table's definition (adds, modifies, or deletes attributes or constraints)
CREATE TABLE AS	Creates a new table based on a query in the user's database schema
DROP TABLE	Permanently deletes a table (and its data)
DROP INDEX	Permanently deletes an index
DROP VIEW	Permanently deletes a view

8

Other SQL commands

OTHER SQL COMMANDS	
COMMAND OR OPTION	DESCRIPTION
Transaction Control Language	
COMMIT	Permanently saves data changes
ROLLBACK	Restores data to its original values
Data Control Language	
GRANT	Gives a user permission to take a system action or access a data object
REVOKE	Removes a previously granted permission from a user

9



SQL Queries

- The **heart** of SQL is the **query**
- In the SQL environment, the word query covers both **questions** and **actions**
- *"What products currently held in inventory are priced over \$100, and what is the quantity on hand for each of those products?"*
- *"How many employees have been hired since January 1, 2016, by each of the company's departments?"*

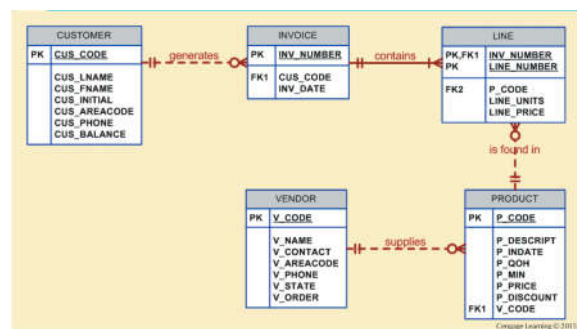
10

SQL Queries

- **Data retrieval** is done in SQL using a **SELECT** query
- Most SQL data manipulation commands operate over an entire table (relation), which is why **SQL commands** are said to be **set-oriented** commands.
- A SQL set-oriented command works over a set of rows

11

Figure 7.1 - The Database Model



12

12

The database model in Figure 7.1 reflects the following **business rules**

- A customer may generate many invoices. Each invoice is generated by one customer.
- An invoice contains one or more invoice lines. Each invoice line is associated with one invoice.
- Each invoice line references one product. A product may be found in many invoice lines. (You can sell more than one hammer to more than one customer.)

13

The database model in Figure 7.1 reflects the following **business rules**

- A vendor may supply many products. Some vendors do not yet supply products. For example, a vendor list may include potential vendors.
- If a product is vendor-supplied, it is supplied by only a single vendor.
- Some products are not supplied by a vendor. For example, some products may be produced in-house or bought on the open market.

14

2. Basic SELECT Queries

- **Each clause** in a SELECT query performs a **specific function**
- **SELECT**—specifies the attributes to be returned by the query
- **FROM**—specifies the table(s) from which the data will be retrieved
- **WHERE**—filters the rows of data based on provided **criteria**
- **GROUP BY**—groups the rows of data into collections based on sharing the same values in one or more attributes
- **HAVING**—filters the groups formed in the GROUP BY clause based on provided criteria
- **ORDER BY**—sorts the final query result rows in ascending or descending order based on the values of one or more attributes

15

Basic SELECT Queries

- Complex command sequences are best shown on **separate lines**, with space between the SQL command and the command's components.
- For a SELECT query to retrieve data from the database, it will require at least a **SELECT column** list and a FROM clause.
- The **SELECT** column list specifies the relational **projection**, as discussed in Chapter 3

16

3. SELECT Statement options

- The SELECT query specifies the columns to be retrieved as a column list.
- The syntax for a basic SELECT query that retrieves data from a table is:

```
SELECT columnlist
FROM tablelist;
```
- The columnlist represents one or more attributes, separated by commas.
- Example:

```
SELECT *
FROM PRODUCT;
```

17

Wildcard character

- A symbol that can be used as a general substitute for:
 - (1) all columns in a table (*) when used in an attribute list of a SELECT statement
 - (2) zero or more characters in a SQL LIKE clause condition (% and _)

18

Figure 7.2 SELECT AN ENTIRE TABLE

```
SELECT *
FROM PRODUCT;
```

Fig7-02

P_CODE	P_DESCRIPTION	P_INDATE	P_QOH	P_MIN	P_PRICE	P_DISCOUNT	V_CODE
11QER/31	Power painter, 15 psi., 3-nozzle	03-Thg11-17	8	5	109.99	0.00	25595
13-Q2/P2	7.25-in. pwr. saw blade	13-Thg12-17	32	15	14.99	0.05	21344
14-Q1/L3	9.00-in. pwr. saw blade	13-Thg11-17	18	12	17.49	0.00	21344
1546-QQ2	Hrd. cloth, 1/4-in., 2x50	15-Thg1-18	15	8	39.95	0.00	23119
1558-QW1	Hrd. cloth, 1/2-in., 3x50	15-Thg1-18	23	5	43.99	0.00	23119
2232/QTY	B&D jigsaw, 12-in. blade	30-Thg12-17	8	5	109.92	0.05	24288
2232/QWE	B&D jigsaw, 8-in. blade	24-Thg12-17	6	5	99.87	0.05	24288
2238/QPD	B&D cordless drill, 1/2-in.	20-Thg1-18	12	5	38.95	0.05	25595
23109-HB	Claw hammer	20-Thg1-18	23	10	9.95	0.10	21225
23114-AA	Sledge hammer, 12 lb.	02-Thg1-18	8	5	14.40	0.05	
54778-2T	Rat-tail file, 1/8-in. fine	15-Thg12-17	43	20	4.99	0.00	21344
89-WRE-Q	Hicut chain saw, 16 in.	07-Thg2-18	11	5	256.99	0.05	24288
PVC23DRT	PVC pipe, 3.5-in., 8-ft	20-Thg2-18	188	75	5.87	0.00	
SM-18277	1.25-in. metal screw, 25	01-Thg3-18	172	75	6.99	0.00	21225
SW-23116	2.5-in. wd. screw, 50	24-Thg2-18	237	100	8.45	0.00	21231
WR3/TT3	Steel matting, 4x8x1/8", 5' mes	17-Thg1-18	18	5	119.95	0.10	25595
			0	0	0.00	0.00	0

19

FIGURE 7.3 SELECT WITH A COLUMN LIST

```
SELECT P_CODE, P_DESCRIPTION, P_PRICE, P_QOH
FROM PRODUCT;
```

Fig7-03

P_CODE	P_DESCRIPTION	P_PRICE	P_QOH
11QER/31	Power painter, 15 psi., 3-nozzle	109.99	8
13-Q2/P2	7.25-in. pwr. saw blade	14.99	32
14-Q1/L3	9.00-in. pwr. saw blade	17.49	18
1546-QQ2	Hrd. cloth, 1/4-in., 2x50	39.95	15
1558-QW1	Hrd. cloth, 1/2-in., 3x50	43.99	23
2232/QTY	B&D jigsaw, 12-in. blade	109.92	8
2232/QWE	B&D jigsaw, 8-in. blade	99.87	6
2238/QPD	B&D cordless drill, 1/2-in.	38.95	12
23109-HB	Claw hammer	9.95	23
23114-AA	Sledge hammer, 12 lb.	14.40	8
54778-2T	Rat-tail file, 1/8-in. fine	4.99	43
89-WRE-Q	Hicut chain saw, 16 in.	256.99	11
PVC23DRT	PVC pipe, 3.5-in., 8-ft	5.87	188
SM-18277	1.25-in. metal screw, 25	6.99	172

20

Using Column Aliases

- Recall that the **attribute** within an **entity** is implemented as a **column** in the table.
- The attribute name becomes the name of that column.
- If the programmer wants a different name to be used as the label in the output, a new name can be specified. The new name is referred to as an **alias**.

21

Using Column Aliases

```
SELECT P_CODE, P_DESCRIPTION AS DESCRIPTION,
P_PRICE AS [Unit Price], P_QOH AS QTY
FROM PRODUCT;
```

Fig7-04

P_CODE	DESCRIPTION	Unit Price	QTY
11QER/31	Power painter, 15 psi., 3-nozzle	109.99	8
13-Q2/P2	7.25-in. pwr. saw blade	14.99	32
14-Q1/L3	9.00-in. pwr. saw blade	17.49	18
1546-QQ2	Hrd. cloth, 1/4-in., 2x50	39.95	15
1558-QW1	Hrd. cloth, 1/2-in., 3x50	43.99	23
2232/QTY	B&D jigsaw, 12-in. blade	109.92	8
2232/QWE	B&D jigsaw, 8-in. blade	99.87	6
2238/QPD	B&D cordless drill, 1/2-in.	38.95	12
23109-HB	Claw hammer	9.95	23
23114-AA	Sledge hammer, 12 lb.	14.40	8
54778-2T	Rat-tail file, 1/8-in. fine	4.99	43
89-WRE-Q	Hicut chain saw, 16 in.	256.99	11
PVC23DRT	PVC pipe, 3.5-in., 8-ft	5.87	188
SM-18277	1.25-in. metal screw, 25	6.99	172

22

Using Column Aliases

- MS Access requires the AS keyword before a column alias.
- MS SQL Server: AS is optional

23

Using Column Aliases

- There are a few things of interest about the use of these aliases:
 - Not all columns in a query must use an alias
 - Aliases that contain a **space** must be inside a delimiter (quotes) or **[]**

24

Using Computed Columns

```
SELECT P_DESCRIPTION, P_QOH, P_PRICE, P_QOH * P_PRICE
FROM PRODUCT;
```

Fig7-05

P_DESCRIPTION	P_QOH	P_PRICE	Expr1
Power painter, 15 psi., 3-nozzle	8	109.99	879.919982910156
7.25-in. pwr. saw blade	32	14.99	479.679992675781
9.00-in. pwr. saw blade	18	17.49	314.820007324219
Hrd. cloth, 1/4-in., 2x50	15	39.95	599.25
Hrd. cloth, 1/2-in., 3x50	23	43.99	1011.77001953125
B&D jigsaw, 12-in. blade	8	109.92	879.359985351563
B&D jigsaw, 8-in. blade	6	99.87	599.220031738281
B&D cordless drill, 1/2-in.	12	38.95	467.400024414063
Claw hammer	23	9.95	228.849990844727
Sledge hammer, 12 lb.	8	14.40	115.199996948242
Rat-tail file, 1/8-in. fine	43	4.99	214.56999206543
Hicut chain saw, 16 in.	11	256.99	2826.88989257813
PVC pipe, 3.5-in., 8-ft	188	5.87	1103.55993652344
1.25-in. metal screw, 25	172	6.99	1202.27990722656
2.5-in. wd. screw, 50	237	8.45	2002.64990234375

25

Using Computed Columns

- Note:
- MS Access** automatically adds an Expr label to all computed columns when an alias is not specified. (The first computed column would be labeled Expr1; the second, Expr2; and so on.)
- Oracle** uses the actual formula text as the label for the computed column.
- Other DBMSs** return the column without a heading label.

26

Using Computed Columns

- To make the output more readable, an alias is typically used for any computed fields.
- For example, you can rewrite the previous SQL statement as follows:

```
SELECT P_DESCRIPTION, P_QOH, P_PRICE, P_QOH * P_PRICE AS
TOTALVALUE
FROM PRODUCT;
```

FIGURE 7.6 SELECT STATEMENT WITH A COMPUTED COLUMN AND AN ALIAS

P_DESCRIPTION	P_QOH	P_PRICE	TOTALVALUE
Power painter, 15 psi., 3-nozzle	8	109.99	879.92
7.25-in. pwr. saw blade	32	14.99	479.68
9.00-in. pwr. saw blade	18	17.49	314.82
Hrd. cloth, 1/4-in., 2x50	15	39.95	599.25
Hrd. cloth, 1/2-in., 3x50	23	43.99	1011.77
B&D jigsaw, 12-in. blade	8	109.92	879.36
B&D jigsaw, 8-in. blade	6	99.87	599.22
B&D cordless drill, 1/2-in.	12	38.95	467.40

27

Arithmetic Operators: The Rule of Precedence

TABLE 7.4

THE ARITHMETIC OPERATORS

OPERATOR	DESCRIPTION
+	Add
-	Subtract
*	Multiply
/	Divide
^	Raise to the power of (some applications use ** instead of ^)

1. Perform operations within parentheses.
2. Perform power operations.
3. Perform multiplications and divisions.
4. Perform additions and subtractions

28

Date Arithmetic

- Because the values are stored as a number of days, it is possible to perform date arithmetic in a query.
- For example, if today's date in some DBMS is the day number "250,000," then tomorrow will be "250,001," and yesterday was "249,999."
- Adding or subtracting a number from a date that is stored in a date data type returns the date that is the specified number of days from the given date.
- Subtracting one date value from another yields the number of days between those dates.

29

Date Arithmetic

- Suppose that a manager wants a list of all products, the dates they were received, and the warranty expiration date (90 days from receiving the product).

```
SELECT P_CODE, P_INDATE, P_INDATE + 90 AS EXPDATE
FROM PRODUCT;
```

30

Listing Unique Values

```
SELECT DISTINCT V_CODE  
FROM PRODUCT;
```

FIGURE 7.7 A LISTING OF DISTINCT V_CODE VALUES IN THE PRODUCT TABLE

V_CODE
21225
21231
21344
23119
24288
25595

31

JOIN Introduce one

```
SELECT CUSTOMER.CUS_CODE, CUS_LNAME,  
INV_NUMBER, INV_DATE  
FROM CUSTOMER INNER JOIN INVOICE  
ON CUSTOMER.CUS_CODE=INVOICE.CUS_CODE;
```

32

4. FROM Clause Options

- INNER JOINS
- OUTER JOINS
 - LEFT OUTER JOINS and
 - RIGHT OUTER JOINS
- The Cartesian product

33

JOIN

- **NATURAL JOIN**
`SELECT column-list`
`FROM table1 NATURAL JOIN table2`
- **JOIN USING**
`SELECT column-list`
`FROM table1 JOIN table2 USING (common-column)`

Oracle and MySQL support the JOIN USING syntax.
MS SQL Server and Access do not

34

JOIN ON Syntax

Syntax:

```
SELECT column-list  
FROM table1 JOIN table2 ON join-condition
```

35

INNER JOIN

Two tables

```
SELECT CUSTOMER.CUS_CODE, CUS_LNAME, INV_NUMBER,  
INV_DATE  
FROM CUSTOMER INNER JOIN INVOICE  
ON CUSTOMER.CUS_CODE=INVOICE.CUS_CODE;
```

Fig7-09

CUS_CODE	CUS_LNAME	INV_NUMBER	INV_DATE
10011	Dunne	1002	16-Thg1-18
10011	Dunne	1004	17-Thg1-18
10011	Dunne	1008	17-Thg1-18
10012	Smith	1003	16-Thg1-18
10014	Orlando	1001	16-Thg1-18
10014	Orlando	1006	17-Thg1-18
10015	O'Brian	1007	17-Thg1-18
10018	Farriss	1005	17-Thg1-18
*			

36

INNER JOIN

Two tables

```
SELECT P_CODE, P_DESCRIPTOR, VENDOR.V_CODE, V_NAME,
V_AREACODE, V_PHONE
FROM VENDOR INNER JOIN PRODUCT
ON VENDOR.V_CODE = PRODUCT.V_CODE;
```

P_CODE	P_DESCRIPTOR	V_CODE	V_NAME	V_AREACODE	V_PHONE
23109-HB	Claw hammer	21225	Bryson, Inc.	615	223-3234
SM-18277	1.25-in. metal screw, 25	21225	Bryson, Inc.	615	223-3234
SW-23116	2.5-in. wd. screw, 50	21231	D&E Supply	615	228-3245
13-Q2/P2	7.25-in. pwr. saw blade	21344	Gomez Bros.	615	889-2546
14-Q1/L3	9.00-in. pwr. saw blade	21344	Gomez Bros.	615	889-2546
54778-2T	Rat-tail file, 1/8-in. fine	21344	Gomez Bros.	615	889-2546
1546-QQ2	Hrd. cloth, 1/4-in., 2x50	23119	Randsets Ltd.	901	678-3998
1558-QW1	Hrd. cloth, 1/2-in., 3x50	23119	Randsets Ltd.	901	678-3998
2232/QTY	B&D jigsaw, 12-in. blade	24288	ORDVA, Inc.	615	898-1234
2232/QWE	B&D jigsaw, 8-in. blade	24288	ORDVA, Inc.	615	898-1234
89-WRE-Q	Hicut chain saw, 16 in.	24288	ORDVA, Inc.	615	898-1234

37

INNER JOIN

Three tables

```
SELECT INVOICE.INV_NUMBER, LINE.P_CODE,
PRODUCT.P_DESCRIPTOR, LINE.LINE_UNITS, LINE.LINE_PRICE
FROM PRODUCT INNER JOIN
(INVOICE INNER JOIN LINE
ON INVOICE.INV_NUMBER = LINE.INV_NUMBER)
ON PRODUCT.P_CODE = LINE.P_CODE;
```

INV_NUMBER	P_CODE	P_DESCRIPTOR	LINE_UNITS	LINE_PRICE
1001	13-Q2/P2	7.25-in. pwr. saw blade	1	14.99
1001	23109-HB	Claw hammer	1	9.95
1002	54778-2T	Rat-tail file, 1/8-in. fine	2	4.99
1003	2238/QPD	B&D cordless drill, 1/2-in.	1	38.95
1003	1546-QQ2	Hrd. cloth, 1/4-in., 2x50	1	39.95
1003	13-Q2/P2	7.25-in. pwr. saw blade	5	14.99
1004	54778-2T	Rat-tail file, 1/8-in. fine	3	4.99
1004	23109-HB	Claw hammer	2	9.95
1005	PVC23DRT	PVC pipe, 3.5-in., 8-ft	12	5.87
1005	SM-18277	1.25-in. metal screw, 25	3	6.89

38

OUTER JOIN

- LEFT OUTER JOIN
- RIGHT OUTER JOIN

39

LEFT JOIN

```
SELECT column-list
FROM table1 LEFT [OUTER] JOIN table2
ON join-condition
```

40

LEFT JOIN

- For example, the following query lists the product code, vendor code, and vendor name for **all vendor** and **includes those vendors with no matching products**:

LEFT OUTER JOIN

```
SELECT PRODUCT.P_CODE,
VENDOR.V_CODE,
VENDOR.V_NAME
FROM VENDOR LEFT JOIN
PRODUCT
ON VENDOR.V_CODE =
PRODUCT.V_CODE;
```

P_CODE	V_CODE	V_NAME
23109-HB	21225	Bryson, Inc.
SM-18277	21225	Bryson, Inc.
	21226	SuperLoo, Inc.
SW-23116	21231	D&E Supply
13-Q2/P2	21344	Gomez Bros.
14-Q1/L3	21344	Gomez Bros.
54778-2T	21344	Gomez Bros.
	22567	Dome Supply
1546-QQ2	23119	Randsets Ltd.
1558-QW1	23119	Randsets Ltd.
	24004	Brackman Bros.
2232/QTY	24288	ORDVA, Inc.
2232/QWE	24288	ORDVA, Inc.
89-WRE-Q	24288	ORDVA, Inc.
	25443	B&K, Inc.
	25501	Damal Supplies
11QER/31	25595	Rubicon Systems
2238/QPD	25595	Rubicon Systems
WR3/TT3	25595	Rubicon Systems

41

42

RIGHT OUTER JOIN

```
SELECT column-list
FROM table1 RIGHT [OUTER] JOIN table2
ON join-condition
```

43

RIGHT OUTER JOIN

- For example, the following query lists the product code, vendor code, and vendor name for **all products** and *includes products that do not have a matching vendor code*:

44

RIGHT OUTER JOIN

```
SELECT
PRODUCT.P_CODE,
VENDOR.V_CODE,
VENDOR.V_NAME
FROM VENDOR RIGHT
JOIN PRODUCT
ON VENDOR.V_CODE =
PRODUCT.V_CODE;
```



P_CODE	V_CODE	V_NAME
23114-AA		
PVC23DRT		
23109-HB	21225	Bryson, Inc.
SM-18277	21225	Bryson, Inc.
SW-23116	21231	D&E Supply
13-Q2/P2	21344	Gomez Bros.
14-Q1/L3	21344	Gomez Bros.
54778-2T	21344	Gomez Bros.
1546-QQ2	23119	Randsets Ltd.
1558-QW1	23119	Randsets Ltd.
2232/QTY	24288	ORDVA, Inc.
2232/QWE	24288	ORDVA, Inc.
89-WRE-Q	24288	ORDVA, Inc.
11QER/31	25595	Rubicon Systems
2238/QPD	25595	Rubicon Systems
WR3/TT3	25595	Rubicon Systems

45

FULL OUTER JOIN

- There is not FULL JOIN on Access
- UNION 2 results from LEFT JOIN and RIGHT JOIN

```
SELECT PRODUCT.P_CODE, VENDOR.V_CODE, VENDOR.V_NAME
FROM VENDOR RIGHT JOIN PRODUCT ON VENDOR.V_CODE =
PRODUCT.V_CODE
UNION
SELECT PRODUCT.P_CODE, VENDOR.V_CODE, VENDOR.V_NAME
FROM VENDOR LEFT JOIN PRODUCT ON VENDOR.V_CODE =
PRODUCT.V_CODE;
```

46

Some where

```
SELECT P_CODE, VENDOR.V_CODE, V_NAME
FROM VENDOR FULL JOIN PRODUCT
ON VENDOR.V_CODE = PRODUCT.V_CODE;
```

47

Cross Join

- A **cross join** performs a relational product (also known as the *Cartesian product*) of two tables.
- The cross join syntax is:
SELECT column-list
FROM table1 CROSS JOIN table2

48

Cross Join

- Unlike Oracle, MS SQL Server, and MySQL, **Access** does not support the CROSS JOIN operator.
- However, all DBMS support producing a cross join by **placing a comma “,”** between the tables in the FROM clause
- Which is the more common method for producing a cross join.

49

Joining Tables with an Alias

```
SELECT P_DESCRPT, P_PRICE, V_NAME,
       V_CONTACT, V_AREACODE,
       V_PHONE
FROM PRODUCT P INNER JOIN VENDOR V
ON P.V_CODE = V.V_CODE;
```

50

Using Table Aliases

- Oracle and MySQL do not use the AS keyword for a table alias,
- while MS SQL Server and Access will accept table aliases either with or without the AS keyword.

51

Recursive Joins

- A **table alias** is especially useful when a table must be joined to itself in a **recursive query**, as is the case when working with unary relationships.

52

Recursive Joins

- Suppose that you are working with the EMP table

EMP_NUM	EMP_TITLE	EMP_LNAME	EMP_FNAME	EMP_INIT	EMP_DOB	EMP_HIRE	EMP_AREA	EMP_PHON	EMP_MGR
100	Mr.	Kolmycz	George	D	15-Thg6-1942	15-Thg3-85 615		324-5456	
101	Ms.	Lewis	Rhonda	G	19-Thg3-85	25-Thg4-86 615		324-4472	100
102	Mr.	Vandam	Rhett		14-Thg11-58	20-Thg12-90 901		675-8993	100
103	Ms.	Jones	Anne	M	16-Thg10-74	28-Thg8-94 615		896-3456	100
104	Mr.	Lange	John	P	08-Thg11-71	20-Thg10-94 901		504-4430	105
105	Mr.	Williams	Robert	D	14-Thg3-75	08-Thg11-98 615		890-3220	
106	Mrs.	Smith	Jeanne	K	12-Thg2-68	05-Thg1-89 615		324-7883	105
107	Mr.	Diante	Jorge	D	21-Thg8-74	02-Thg7-94 615		890-4567	105
108	Mr.	Wiesenbach	Paul	R	14-Thg2-86	18-Thg11-92 615		897-4358	
109	Mr.	Smith	George	K	18-Thg5-61	14-Thg4-89 901		504-3339	108
110	Mrs.	Genkazi	Leigha	W	19-Thg5-70	01-Thg12-90 901		569-0093	108
111	Mr.	Washington	Rupert	E	03-Thg1-66	21-Thg6-93 615		890-4925	105
112	Mr.	Johnson	Edward	E	14-Thg5-61	01-Thg12-83 615		898-4387	100
113	Ms.	Smythe	Melanie	P	15-Thg5-70	11-Thg5-99 615		324-9006	105
114	Ms.	Brandon	Maie	G	02-Thg11-56	15-Thg11-79 901		882-0845	108
115	Mrs.	Saranda	Hermine	R	25-Thg7-72	23-Thg4-93 615		324-5505	105
116	Mr.	Smith	George	A	08-Thg11-65	10-Thg12-88 615		890-2984	108
0									0

53

Recursive Joins

- Using the data in the EMP table, you can generate a list of all employees with their managers' names by joining the EMP table to itself.

```
SELECT E.EMP_NUM, E.EMP_LNAME,
       E.EMP_MGR, M.EMP_LNAME
FROM EMP AS E INNER JOIN EMP AS M
ON M.EMP_NUM = E.EMP_MGR;
```

EMP_N	E.EMP_LNAM	EMP_M	M.EMP_LNAME
101	Lewis	100	Kolmycz
102	Vandam	100	Kolmycz
103	Jones	100	Kolmycz
104	Lange	105	Williams
106	Smith	105	Williams
107	Diante	105	Williams
109	Smith	108	Wiesenbach
110	Genkazi	108	Wiesenbach
111	Washington	105	Williams
112	Johnson	100	Kolmycz
113	Smythe	105	Williams
114	Brandon	108	Wiesenbach
115	Saranda	105	Williams
116	Smith	108	Wiesenbach

54

5. ORDER BY Clause Options

- The ORDER BY clause is especially useful when the listing order is important to you.
- The syntax is:

```
SELECT columnlist
FROM tablelist
[ORDER BY columnlist [ASC | DESC] ];
```

Example:

```
SELECT P_CODE, P_DESCRIPT, P_QOH, P_PRICE
FROM PRODUCT
ORDER BY P_PRICE;
```

DESC

```
SELECT P_CODE, P_DESCRIPT, P_QOH, P_PRICE
FROM PRODUCT
ORDER BY P_PRICE DESC;
```

55

56

ORDER BY

Ordered listings are used frequently. For example, suppose that you want to create a phone directory. It would be helpful if you could produce an ordered sequence (last name, first name, initial) in three stages:

- 1. ORDER BY last name.
- 2. Within matching last names, ORDER BY first name.
- 3. Within matching first and last names, ORDER BY middle initial.

Such a multilevel ordered sequence is known as a cascading order sequence, and it can be created easily by listing several attributes, separated by commas, after the ORDER BY clause.

The cascading order sequence is the basis for any telephone directory. To illustrate a cascading order sequence, use the following SQL command on the EMPLOYEE table:

```
SELECT EMP_LNAME, EMP_FNAME, EMP_INITIAL, EMP_AREACODE,
EMP_PHONE
FROM EMPLOYEE
ORDER BY EMP_LNAME, EMP_FNAME, EMP_INITIAL;
```

57

ORDER BY

FIGURE 7.19 TELEPHONE LIST QUERY RESULTS

EMP_LNAME	EMP_FNAME	EMP_INITIAL	EMP_AREACODE	EMP_PHONE
Brandon	Marie	G	901	882-0845
Diante	Jorge	D	615	890-4567
Genkazi	Leighla	W	901	569-0093
Johnson	Edward	E	615	898-4387
Jones	Anne	M	615	898-3456
Kolmycz	George	D	615	324-5456
Lange	John	P	901	504-4430
Lewis	Rhonda	G	615	324-4472
Saranda	Hermine	R	615	324-5505
Smith	George	A	615	890-2984
Smith	George	K	901	504-3339
Smith	Jeanine	K	615	324-7883
Smythe	Melanie	P	615	324-8006
Vandam	Rhett		901	675-8993
Washington	Rupert	E	615	890-4925
Wiesenbach	Paul	R	615	897-4358
Williams	Robert	D	615	890-3220

58

ORDER BY

```
SELECT P_CODE, P_DESCRIPT, V_CODE,
P_PRICE * P_QOH AS TOTAL
FROM PRODUCT
ORDER BY V_CODE, P_PRICE * P_QOH DESC;
```

59

FIGURE 7.20 ORDERING BY A DERIVED ATTRIBUTE

FIGURE 7.20 ORDERING BY A DERIVED ATTRIBUTE

P_CODE	P_DESCRIPT	V_CODE	TOTAL
PVC23DRT	PVC pipe, 3.5-in., 8-ft		1103.56
23114-AA	Sledge hammer, 12 lb.		115.20
SM-18277	1.25-in. metal screw, 25	21225	1202.28
23109-HB	Claw hammer	21225	228.85
SW-23116	2.5-in. wvd. screw, 50	21231	2002.65
13-Q2IP2	7.25-in. pwr. saw blade	21344	479.68
14-Q1A3	9.00-in. pwr. saw blade	21344	314.82
54778-2T	Rat-tail file, 1/8-in. fine	21344	214.57
1558-QW1	Hrd. cloth, 1/2-in., 3x50	23119	1011.77
1546-QQ2	Hrd. cloth, 1/4-in., 2x50	23119	599.25
89-WRE-Q	Hicut chain saw, 16 in.	24288	2826.89
2232QTY	B&D jigsaw, 12-in. blade	24288	879.36
2232QWE	B&D jigsaw, 8-in. blade	24288	599.22
WR3/TT3	Steel matting, 4'x8'x1/8", 5" mesh	25595	2159.10
11QER/31	Power painter, 15 psi., 3-nozzle	25595	879.92
2238QPD	B&D cordless drill, 1/2-in.	25595	467.40

60

6. WHERE Clause Options

- Selecting Rows with Conditional Restrictions
- Using Comparison Operators on Character Attributes
- Using Comparison Operators on Dates
- Logical Operators: AND, OR, and NOT
- Special Operators
 - BETWEEN, IN, LIKE, IS NULL

61

Selecting Rows with Conditional Restrictions

```
SELECT columnlist
FROM tablelist
[WHERE conditionlist ]
[ORDER BY columnlist [ASC | DESC] ];
```

62

Comparison operators

TABLE 7.6	
COMPARISON OPERATORS	
SYMBOL	MEANING
=	Equal to
<	Less than
<=	Less than or equal to
>	Greater than
>=	Greater than or equal to
<> or !=	Not equal to

63

Examples

```
SELECT P_DESCRIPTOR, P_QOH, P_PRICE, V_CODE
FROM PRODUCT
WHERE V_CODE = 21344;
```

```
SELECT P_DESCRIPTOR, P_QOH, P_PRICE, V_CODE
FROM PRODUCT
WHERE V_CODE <> 21344;
```

```
SELECT P_DESCRIPTOR, P_QOH, P_MIN, P_PRICE
FROM PRODUCT
WHERE P_PRICE <= 10;
```

64

Logical Operators: AND, OR, and NOT

```
SELECT P_DESCRIPTOR, P_QOH, P_PRICE, V_CODE
FROM PRODUCT
WHERE V_CODE = 21344
OR V_CODE = 24288;
```

```
SELECT P_DESCRIPTOR, P_QOH, P_PRICE, V_CODE
FROM PRODUCT
WHERE P_PRICE > 100
AND P_QOH < 20;
```

FIGURE 7.28 THE LOGICAL AND

P_DESCRIPTOR	P_QOH	P_PRICE	V_CODE
Power painter, 15 psi., 3-nozzle	8	109.99	2559
B&D jigsaw, 12-in. blade	8	109.92	2428
Hicof chain saw, 16 in.	11	256.99	2428
Steel matting, 4'x8'x1/8", 5" mesh	18	119.95	2559

65

Logical Operators: AND, OR, and NOT

```
SELECT P_DESCRIPTOR, P_PRICE, V_CODE
FROM PRODUCT
WHERE (V_CODE = 25595 OR V_CODE = 24288)
AND P_PRICE > 100;
```

```
SELECT *
FROM PRODUCT
WHERE NOT (V_CODE = 21344);
```

66

Using Comparison Operators on Character Attributes

```
SELECT    P_CODE, P_DESCRIPT, P_QOH, P_MIN, P_PRICE
FROM      PRODUCT
WHERE     P_CODE < '1558-QW1';
```

Fig7-24

P_DESCRIPT	P_QC	P_M	P_PRK
Claw hammer	23	10	9.95
Rat-tail file, 1/8-in. fine	43	20	4.99
Trứng luộc của trường HUCE	188	75	5.87
1.25-in. metal screw, 25	172	75	6.99
2.5-in. wd. screw, 50	237	100	8.45
*	0	0	0.00

67

Using Comparison Operators on Dates

```
SELECT    P_DESCRIPT, P_QOH, P_MIN, P_PRICE, P_INDATE
FROM      PRODUCT
WHERE     P_INDATE >= #20-Jan-2018#;
```

MS Access users must use the # delimiters for dates.

In MySQL, the expected date format is yyyy-mm-dd, so the WHERE clause would be written as:

```
WHERE     P_INDATE >= '2018-01-20'
```

FIGURE 7.26 SELECTED PRODUCT TABLE ATTRIBUTES: DATE RESTRICTION

P_DESCRIPT	P_QOH	P_MIN	P_PRICE	P_INDATE
B&D cordless drill, 1/2-in.	12	5	38.95	20-Jan-18
Claw hammer	23	10	9.95	20-Jan-18
Hicut chain saw, 16 in.	11	5	256.99	07-Feb-18
PVC pipe, 3.5-in., 8-ft	188	75	5.87	20-Feb-18
1.25-in. metal screw, 25	172	75	6.99	01-Mar-18
2.5-in. wd. screw, 50	237	100	8.45	24-Feb-18

68

Special Operators

- BETWEEN
- IN
- LIKE
- IS NULL

69

The BETWEEN Special Operator

```
SELECT    *
FROM      PRODUCT
WHERE     P_PRICE BETWEEN 50.00 AND 100.00;
```

When using the BETWEEN special operator, always specify the lower-range value first.

The WHERE clause of the command above is interpreted as:

```
WHERE     P_PRICE >= 50 AND P_PRICE <= 100
```

70

The IN Special Operator

```
SELECT    *
FROM      PRODUCT
WHERE     V_CODE = 21344 OR V_CODE = 24288;
```

```
-----
SELECT    *
FROM      PRODUCT
WHERE     V_CODE IN (21344, 24288);
```

71

The LIKE Special Operator

- The **Like** special operator is used in conjunction with wildcards to find patterns within string attributes.
- Some RDBMSs, such as **MS Access**, use the wildcard characters * and ? []
- Others: % and _.
- Example:

```
SELECT    V_NAME, V_CONTACT, V_AREACODE, V_PHONE
FROM      VENDOR
WHERE     VENDOR.V_CONTACT Like "Smith*";
```

72

The LIKE Special Operator

The logical operators may be used in conjunction with the special operators.

For instance, the following query:

```
SELECT V_NAME, V_CONTACT, V_AREACODE, V_PHONE
FROM VENDOR
WHERE V_CONTACT NOT LIKE 'Smith*';
```

73

The LIKE Special Operator

- Standard SQL allows you to use the percent sign (%) and underscore (_) wildcard characters to make matches when the entire string is not known:
- % means any and all *following* or *preceding* characters are eligible. For example:
 - 'J%' includes Johnson, Jones, Jernigan, July, and J-231Q.
 - 'Jo%' includes Johnson and Jones.
 - '%n' includes Johnson and Jernigan.
- _ means any one character may be substituted for the underscore. For example:
 - '_23-456-6789' includes 123-456-6789, 223-456-6789, and 323-456-6789.
 - '_23_56-678_' includes 123-156-6781, 123-256-6782, and 823-956-6788.
 - '_o_es' includes Jones, Cones, Cokes, totes, and roles

74

The IS NULL Special Operator

- To list all products that do not have a vendor assigned (i.e., the V_CODE attribute does not contain a value).

```
SELECT P_CODE, P_DESCRIPT, V_CODE
FROM PRODUCT
WHERE V_CODE IS NULL;
```

P_CODE	P_DESCRIPT	V_CODE
23114-AA	Bánh bao của trường HUCE	
PVC23DRT	Trứng luộc của trường HUCE	

75

FIGURE 7.33 VENDORS not associated with any product

```
SELECT VENDOR.V_CODE, V_NAME, P_CODE
FROM PRODUCT RIGHT JOIN VENDOR
ON PRODUCT.V_CODE = VENDOR.V_CODE
WHERE PRODUCT.P_CODE Is Null;
```

FIGURE 7.33 VENDORS NOT ASSOCIATED WITH ANY PRODUCT

V_CODE	V_NAME	P_CODE
21226	SuperLoo, Inc.	
22567	Dome Supply	
24004	Brackman Bros.	
25443	B&K, Inc.	
25501	Damal Supplies	

76

7. Aggregate Processing

Figure 7.34 total value of products from VENDOR in TN or KY

```
SELECT V.V_CODE, V_NAME, V_STATE, P_CODE, P_DESCRIPT, P_PRICE * P_QOH AS TOTAL
FROM PRODUCT P INNER JOIN VENDOR V ON P.V_CODE = V.V_CODE
WHERE V_STATE IN ('TN', 'KY')
ORDER BY V_STATE, P_PRICE * P_QOH DESC;
```

V_CODE	V_NAME	V_STATE	P_CODE	P_DESCRIPT	TOTAL
21344	Gomez Bros.	KY	13-Q2/P2	7.25-in. pwr. saw blade	479.67999
21344	Gomez Bros.	KY	14-Q1/L3	9.00-in. pwr. saw blade	314.82001
21344	Gomez Bros.	KY	54778-2T	Rat-tail file, 1/8-in. fine	214.56999
24288	ORDVA, Inc.	TN	89-WRE-Q	Hicut chain saw, 16 in.	2826.8899
21231	D&E Supply	TN	SW-23116	2.5-in. wd. screw, 50	2002.6499
21225	Bryson, Inc.	TN	SM-18277	1.25-in. metal screw, 25	1202.2799
24288	ORDVA, Inc.	TN	2232/QTY	B&D jigsaw, 12-in. blade	879.35999
24288	ORDVA, Inc.	TN	2232/QWE	B&D jigsaw, 8-in. blade	599.22003
21225	Bryson, Inc.	TN	23109-HB	Claw hammer	228.84999

77

Aggregate Processing

- The defining characteristic of using an aggregate function is that it takes a collection of rows and reduces it to a single row.
- SQL provides useful aggregate functions that count, find minimum and maximum values, calculate averages, and so on.
- Better yet, SQL allows the user to limit queries to only those entries that have no duplicates or entries whose duplicates can be grouped.

78

7a Aggregate Functions

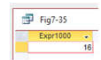
TABLE 7.7

SOME BASIC SQL AGGREGATE FUNCTIONS

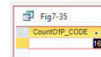
FUNCTION	OUTPUT
COUNT	The number of rows containing non-null values
MIN	The minimum attribute value encountered in a given column
MAX	The maximum attribute value encountered in a given column
SUM	The sum of all values for a given column
AVG	The arithmetic mean (average) for a specified column

COUNT

```
SELECT COUNT(P_CODE)
FROM PRODUCT;
```



```
SELECT Count(PRODUCT.P_CODE) AS CountOfP_CODE
FROM PRODUCT;
```



79

80

COUNT

- The collection of rows does not have to be composed of all of the rows in the table.
- For example, we can use the following query to determine how many products have a price that is less than \$10.

```
SELECT COUNT(P_PRICE)
FROM PRODUCT
WHERE P_PRICE < 10;
```

COUNT

- Aggregate functions take a value, typically an **attribute**, as a **parameter inside parentheses**.
- In Figure 7.35, the code counted the values in the primary key, P_CODE, in the PRODUCT table.
- Notice the difference in the result when counting the V_CODE attribute in that same table, as shown in Figure 7.36.

```
SELECT COUNT(V_CODE)
FROM PRODUCT;
```

FIGURE 7.36 COUNT OF VENDOR CODES IN THE PRODUCT TABLE

CountOfV_CODE
14

81

82

COUNT

- If the purpose of a query is to return the number of rows in a collection, regardless of whether or not any specific column contains nulls, then the syntax COUNT(*) can be used.

COUNT

- COUNT can also be used in conjunction with the DISTINCT clause:

```
SELECT COUNT(*)
FROM (SELECT DISTINCT V_CODE FROM PRODUCT WHERE
V_CODE IS NOT NULL);
```

FIGURE 7.37 COUNT OF DISTINCT VENDOR CODES

Count Distinct
6

83

84

MIN and MAX

```
SELECT MAX(P_PRICE) AS MAXPRICE, MIN(P_PRICE) AS MINPRICE  
FROM PRODUCT;
```

FIGURE 7.38 MAXIMUM AND MINIMUM PRICE OUTPUT

MAXPRICE	MINPRICE
256.99	4.99

85

SUM

```
SELECT SUM(CUS_BALANCE) AS TOTBALANCE  
FROM CUSTOMER;
```

```
SELECT SUM(P_QOH * P_PRICE) AS TOTVALUE  
FROM PRODUCT;
```

FIGURE 7.39 TOTAL VALUE OF ALL ITEMS IN THE PRODUCT TABLE

TOTVALUE
15084.52

86

AVG

```
SELECT AVG(P_PRICE) AS AVGPRICE  
FROM PRODUCT;
```

87

7b Grouping Data

- The aggregate functions summarized data across all rows in the given tables.
- Sometimes, however, you do not want to treat the entire table as a single collection of data for summarizing.
- Rows can be grouped into smaller collections quickly and easily using the GROUP BY clause within the SELECT statement.

88

7b Grouping Data

- The aggregate functions will then summarize the data within each smaller collection.
- The syntax is:

```
SELECT columnlist  
FROM tablelist  
[WHERE conditionlist ]  
[GROUP BY columnlist ]  
[ORDER BY columnlist [ASC | DESC] ];
```

89

7b Grouping Data

- Figure 7.40 determined the average price of all of the products in the database.
- However, what if instead of seeing the price across all products, the users wanted to see the **average price of the products provided by each vendor**? The following query will answer that question, as shown in Figure 7.41.

90

7b Grouping Data

```
SELECT PRODUCT.V_CODE, Avg(PRODUCT.P_PRICE) AS AVGPRICE
FROM PRODUCT
GROUP BY PRODUCT.V_CODE;
```

Aggregate functions ignore nulls when performing calculations, but the GROUP BY clause includes nulls and considers all of the nulls to be the same when forming collections.

Fig7-41

V_CODE	AVGPRICE
	10.13
21225	8.47
21231	8.45
21344	12.49
23119	41.97
24288	155.59
25595	89.63

91

7b Grouping Data

- Understanding the interaction between the **GROUP BY** clause and **aggregate functions** is **crucial** in using them correctly.

```
SELECT PRODUCT.V_CODE, VENDOR.V_NAME,
Count(PRODUCT.P_CODE) AS NUMPRODS,
Avg(PRODUCT.P_PRICE) AS AVGPRICE
FROM VENDOR INNER JOIN PRODUCT
ON VENDOR.V_CODE = PRODUCT.V_CODE
GROUP BY PRODUCT.V_CODE, VENDOR.V_NAME;
```

92

7b Grouping Data

V_CODE	V_NAME	NUMPRODS	AVGPRICE
21225	Bryson, Inc.	1	6.98999977111816
21231	D&E Supply	1	8.44999980926514
21344	Gomez Bros.	3	12.4899997711182
23119	Randsets Ltd.	2	41.9700012207031
24288	ORDVA, Inc.	3	155.593330383301
25595	Rubicon Systems	3	89.6299985249837

93

Edit -> Error

```
SELECT PRODUCT.P_QOH, PRODUCT.V_CODE,
VENDOR.V_NAME, Count(PRODUCT.P_CODE) AS
NUMPRODS, Avg(PRODUCT.P_PRICE) AS AVGPRICE
FROM VENDOR INNER JOIN PRODUCT ON
VENDOR.V_CODE = PRODUCT.V_CODE
GROUP BY PRODUCT.V_CODE, VENDOR.V_NAME;
```

94

Edit

```
SELECT PRODUCT.P_QOH, PRODUCT.V_CODE,
VENDOR.V_NAME, Count(PRODUCT.P_CODE) AS
NUMPRODS, Avg(PRODUCT.P_PRICE) AS AVGPRICE
FROM VENDOR INNER JOIN PRODUCT ON
VENDOR.V_CODE = PRODUCT.V_CODE
GROUP BY PRODUCT.V_CODE, VENDOR.V_NAME,
PRODUCT.P_QOH;
```

95

Result

- The addition of even one attribute can significantly change the results returned by the query.

P_QOH	V_CODE	V_NAME	NUMPRODS	AVGPRICE
172	21225	Bryson, Inc.	1	6.98999977111816
237	21231	D&E Supply	1	8.44999980926514
18	21344	Gomez Bros.	1	17.4899997711182
32	21344	Gomez Bros.	1	14.9899997711182
43	21344	Gomez Bros.	1	4.98999977111816
15	23119	Randsets Ltd.	1	39.9500007629395
23	23119	Randsets Ltd.	1	43.9900016784668
6	24288	ORDVA, Inc.	1	99.870002746582
9	24288	ORDVA, Inc.	1	109.919998168945
11	24288	ORDVA, Inc.	1	256.989990234375
8	25595	Rubicon Systems	1	109.98999786377
12	25595	Rubicon Systems	1	38.9500007629395
18	25595	Rubicon Systems	1	119.949996948242

96

```

SELECT VENDOR.V_CODE, VENDOR.V_NAME, VENDOR.V_STATE,
Count(PRODUCT.P_CODE) AS NUMPRODS, Avg(PRODUCT.P_PRICE) AS
AVGPRICE
FROM VENDOR INNER JOIN PRODUCT ON VENDOR.V_CODE =
PRODUCT.V_CODE
GROUP BY VENDOR.V_CODE, VENDOR.V_NAME, VENDOR.V_STATE;

```

FIGURE 7.45 GROUPS INCLUDING V_STATE

V_CODE	V_NAME	V_STATE	NUMPRODS	AVGPRICE
21225	Bryson, Inc.	TN	2	8.47
21231	D&E Supply	TN	1	8.45
21344	Gomez Bros.	KY	3	12.49
23119	Randsets Ltd.	GA	2	41.97
24288	ORDVA, Inc.	TN	3	155.59
25595	Rubicon Systems	FL	3	89.63

7c HAVING Clause

```

SELECT columnlist
FROM tablelist
[WHERE conditionlist ]
[GROUP BY columnlist ]
[HAVING conditionlist ]
[ORDER BY columnlist [ASC | DESC] ];

```

97

98

Application of the HAVING Clause

```

SELECT V_CODE, COUNT(P_CODE) AS NUMPRODS
FROM PRODUCT
GROUP BY V_CODE
HAVING AVG(P_PRICE) < 10
ORDER BY V_CODE;

```

V_CODE	NUMPRODS
21225	1
21231	1

Note

- If you use the WHERE clause instead of the HAVING clause, the query in Figure 7.46 will produce an error message.
- WHERE is used to restrict rows and is executed prior (**trước**) to the GROUP BY clause.
- Because WHERE executes before GROUP BY, WHERE cannot contain an aggregate function because the collections needed by the aggregate function do not exist yet.
- The HAVING clause is used to restrict groups and is executed after the GROUP BY clause.
- HAVING **can contain aggregate functions** because the collections are formed by the GROUP BY clause before the HAVING is executed. HAVING restricts groups.

99

100

Note

```

SELECT PRODUCT.V_CODE, V_NAME, SUM(P_QOH * P_PRICE) AS
TOTCOST
FROM PRODUCT INNER JOIN VENDOR ON PRODUCT.V_CODE =
VENDOR.V_CODE
WHERE P_DISCOUNT > 0
GROUP BY PRODUCT.V_CODE, V_NAME
HAVING (SUM(P_QOH * P_PRICE) > 500)
ORDER BY SUM(P_QOH * P_PRICE) DESC;

```

V_CODE	V_NAME	TOTCOST
24288	ORDVA, Inc.	4305.46990966797
25595	Rubicon Systems	2626.49987792969

Note

- This statement does the following:
 - Joins the product and vendor tables using V_CODE as the common attribute
 - Restricts to only the rows with a discount greater than 0
 - Groups the remaining rows into collections based on V_CODE and V_NAME
 - Aggregates the total cost of products in each group
 - Restricts to only the groups with totals that exceed \$500
 - Lists the results in descending order by the total cost

101

102

8. Subqueries

- It is often necessary to process data based on other processed data
- For example, suppose that you want to generate a list of vendors who do not provide products. (Recall that not all vendors in the VENDOR table have provided products—some are only *potential vendors*.)

```
SELECT PRODUCT.V_CODE, V_NAME
FROM PRODUCT RIGHT JOIN VENDOR ON PRODUCT.V_CODE =
VENDOR.V_CODE
WHERE P_CODE IS NULL;
```

Subqueries

- This result can also be found by using a subquery.

```
SELECT      V_CODE, V_NAME
FROM        VENDOR
WHERE       V_CODE NOT IN (SELECT V_CODE
                           FROM PRODUCT WHERE V_CODE IS NOT NULL);
```

103

104

Subqueries

- Similarly, to generate a list of all products with a price greater than or equal to the average product price, you can write the following query:

```
SELECT      P_CODE, P_PRICE
FROM        PRODUCT
WHERE       P_PRICE >= (SELECT AVG(P_PRICE) FROM
                        PRODUCT);
```

Subqueries

- In both queries, you needed to get information that was not previously known:
 - What vendors provide products?
 - What is the average price of all products?
- You used a subquery to generate the required information, which could then be used as input for the originating query.

105

106

Subqueries

- There are **key characteristics** that you should remember for subqueries:
 - A subquery is a query (SELECT statement) inside another query.
 - A subquery is normally expressed inside parentheses.
 - The first query in the SQL statement is known as the outer query.
 - The inner query is executed first.
 - The output of an inner query is used as the input for the outer query.
 - The entire SQL statement is sometimes referred to as a nested query.

Subqueries

- The subquery is always on the right side of a comparison or assigning expression.
- Also, a subquery can return one or more values. To be precise, the subquery can return:
 - One single value (one column and one row)
 - A list of values (one column and multiple rows)
 - A virtual table (multicolumn, multirow set of values)
 - subquery can return no values at all, NULL

107

108

Subqueries

- 8a WHERE Subqueries
- 8b IN Subqueries
- 8c HAVING Subqueries
- 8d Multirow Subquery Operators: ALL and ANY
- 8e FROM Subqueries
- 8f Attribute List Subqueries

109

9. SQL Functions

110

10. Relation Set Operators

- UNION
- INTERSECT
- EXCEPT (MINUS)

111

11. Crafting SELECT Queries

- Know Your Data
- Know the Problem
- Build One Clause at a Time

112

SQL data manipulation commands

SQL DATA MANIPULATION COMMANDS	
COMMAND, OPTION, OR OPERATOR	DESCRIPTION
SELECT	Selects attributes from rows in one or more tables or views
FROM	Specifies the tables from which data should be retrieved
WHERE	Restricts the selection of rows based on a conditional expression
GROUP BY	Groups the selected rows based on one or more attributes
HAVING	Restricts the selection of grouped rows based on a condition
ORDER BY	Orders the selected rows based on one or more attributes
INSERT	Inserts row(s) into a table
UPDATE	Modifies an attribute's values in one or more table's rows
DELETE	Deletes one or more rows from a table
Comparison operators	
=, <, >, <=, >=, <>, !=	Used in conditional expressions
Logical operators	
AND/OR/NOT	Used in conditional expressions

113

SQL data manipulation commands

Special operators	Used in conditional expressions
BETWEEN	Checks whether an attribute value is within a range
IN	Checks whether an attribute value matches any value within a value list
LIKE	Checks whether an attribute value matches a given string pattern
IS NULL	Checks whether an attribute value is null
EXISTS	Checks whether a subquery returns any rows
DISTINCT	Limits values to unique values
Aggregate functions	Used with SELECT to return mathematical summaries on columns
COUNT	Returns the number of rows with non-null values for a given column
MIN	Returns the minimum attribute value found in a given column
MAX	Returns the maximum attribute value found in a given column
SUM	Returns the sum of all values for a given column
AVG	Returns the average of all values for a given column

114

SQL data definition commands

SQL DATA DEFINITION COMMANDS	
COMMAND OR OPTION	DESCRIPTION
CREATE SCHEMA	Creates a database schema
AUTHORIZATION	
CREATE TABLE	Creates a new table in the user's database schema
NOT NULL	Ensures that a column will not have null values
UNIQUE	Ensures that a column will not have duplicate values
PRIMARY KEY	Defines a primary key for a table
FOREIGN KEY	Defines a foreign key for a table
DEFAULT	Defines a default value for a column (when no value is given)
CHECK	Validates data in an attribute
CREATE INDEX	Creates an index for a table
CREATE VIEW	Creates a dynamic subset of rows and columns from one or more tables
ALTER TABLE	Modifies a table's definition (adds, modifies, or deletes attributes or constraints)
CREATE TABLE AS	Creates a new table based on a query in the user's database schema
DROP TABLE	Permanently deletes a table (and its data)
DROP INDEX	Permanently deletes an index
DROP VIEW	Permanently deletes a view

115

Other SQL commands

OTHER SQL COMMANDS	
COMMAND OR OPTION	DESCRIPTION
Transaction Control Language	
COMMIT	Permanently saves data changes
ROLLBACK	Restores data to its original values
Data Control Language	
GRANT	Gives a user permission to take a system action or access a data object
REVOKE	Removes a previously granted permission from a user

116